Ejercicios Programación Orientada a Objetos.

En esta guía encontrarás una serie de prácticas para ser realizadas de la mano de la unidad POO inicial de Programación III. Repasá el contenido cuando sea necesario y cualquier consulta, no dudes en preguntar.

1) Portón Automático (caso incremental en 3 pasos)

Clase: PortonAutomatico

Propiedades obligatorias:

- int Apertura (valores 0 a 100; 0=cerrado; 100=abierto completo)
- bool EstaCerrado (derivado: Apertura == 0)
- bool EstaAbiertoCompleto (derivado: Apertura == 100)

Cuando se abre completo, la propiead abierto completo será true, de lo contrario será false. Cuando está cerrado, la propiedad EstadoCerrado será true, de lo contrario será false.

Constructores:

- Vacío (inicia cerrado).
- Con int aperturaInicial (validar 0–100. Si es menos de cero, se setea cero. Si es más de 100, se setea 100).

Parte A: Métodos obligatorios:

- void Abrir(): deja en 100 si no está ya en 100.
- void Cerrar(): deja en 0 si no está ya en 0.
- void MostrarEstado(): imprime “Cerrado”, “Abierto total (100%)” o “Abierto parcial (X%)”.

Validaciones/mensajes:

- Si ya está en 100 y se llama Abrir(): mostrar “Ya está abierto al 100%”.
- Si ya está en 0 y se llama Cerrar(): mostrar “Ya está completamente cerrado”.

Parte B – peatonal (sobrecarga):

- void Abrir(int porcentaje): abre al % indicado (validar 1–99).
- void AbrirPeatonal(): atajo que deje en 20% abierto.

Mensajes: si el porcentaje es inválido: “Porcentaje inválido. Debe ser 1–99.”

Parte C – mejoras:

- void Stop(): no cambia el valor, solo imprime “Movimiento detenido” (simulación).
- bool Toggle(): si está cerrado, abre total y devuelve true; si no, cierra y devuelve false.

Nota: en todos los casos se actualizan las propiedades de estado para chequear.

Pruebas mínimas:

1. Iniciar cerrado, Abrir() → 100 → “Abierto total (100%)”.
2. AbrirPeatonal() desde 100 → pasar a 20 y mostrar “Abierto parcial (20%)”.
3. Cerrar() → 0.
4. Abrir(105) → “Porcentaje inválido. Debe ser 1–99”.
5. Simulaciones: Abrir(10) con un ciclo e ir consultando el estado. Cuando llegue a 50% ejecutar Stop() y terminar.

2) Cuenta Bancaria

Clase: CuentaBancaria

Propiedades: string NumeroCuenta, string Titular, decimal Saldo

Constructores: vacío y con valores iniciales (Saldo ≥ 0).

Métodos:

- void Depositar(decimal monto): valida monto > 0.
- bool Extraer(decimal monto): si monto <= Saldo, descuenta y devuelve true; si no, **no** descuenta y devuelve false.
- void MostrarEstado(): número, titular y saldo.

Validaciones/mensajes:

- Depósito con monto <= 0: “Monto inválido.”
- Extracción sin fondos: **imprimir** “Saldo insuficiente.” y no modificar el saldo.

Pruebas mínimas:

- Depositar 10.000, extraer 12.000: “Saldo insuficiente.”, saldo intacto.
- Extraer 5.000: ok.
- MostrarEstado para ver resultado.

3) Cafetera

Clase: Cafetera

Propiedades: int CapacidadMl, int NivelMl

Constructores:

- Vacío: CapacidadMl=1000, NivelMl=0.
- Con capacidadInicial y nivelInicial (validar 0 ≤ nivel ≤ capacidad).

Métodos:

- void CargarAgua(int ml): suma al nivel (sin exceder).
- int ServirTaza(): sirve **200 ml por defecto** (si alcanza); devuelve ml realmente servidos.
- **Sobrecarga:** int ServirTaza(int ml): sirve cantidad pedida si alcanza.
- void MostrarEstado(): "Nivel: X/Y ml".

Validaciones/mensajes:

- Si no alcanza: "No hay suficiente café para servir esa cantidad."
- Si al cargar excede: "Capacidad superada. Se cargó hasta el máximo."

Pruebas mínimas:

- Cargar 500 ml, servir taza default (200), mostrar 300 restantes.
- Intentar servir 400: mensaje de insuficiente, sin cambios.